

Day 05

MINI PROJECT – SIMPLE DATA CLEANING PIPELINE

STEP 1 – Create Project Folder (Linux)

Open terminal.

Create a folder called:

student_data_project

Inside it create files: students.csv, clean_students.csv, analysis.sql, process_data.py

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python$ cd student_data_project/
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/student_data_project$ touch students.csv clean_students.csv analysis.sql process_data.py
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/student_data_project$ ls -lrth
total 0
-rwxrwxrwx 1 prashant prashant 0 Mar 10 2026 students.csv
-rwxrwxrwx 1 prashant prashant 0 Mar 10 2026 clean_students.csv
-rwxrwxrwx 1 prashant prashant 0 Mar 10 2026 process_data.py
-rwxrwxrwx 1 prashant prashant 0 Mar 10 2026 analysis.sql
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/student_data_project$ |
```

STEP 2 – Create Dataset

Create a CSV file called students.csv.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/student_data_project$ cat students.csv | column -t -s,
id  name  marks city
1   Rahul  85    Mumbai
2   Sneha  78    Pune
3   Amit   90    Delhi
4   Priya  65    Nashik
5   Rohit  88    Mumbai
6   Anjali 72    Pune
7   Karan  95    Delhi
8   Meena  60    Nashik
9   Vijay  81    Mumbai
10  Pooja  75    Pune
11  Arun   69    Delhi
12  Neha   84    Nashik
13  Sanjay 77    Mumbai
14  Kavita 91    Pune
15  Deepak 73    Delhi
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/student_data_project$ |
```

STEP 3 – Python Task

Create a Python script called:

process_data.py

Tasks

1. Read the students.csv file.
2. Remove students whose marks are less than 70.
3. Convert student names to uppercase.
4. Save the cleaned data into:

clean_students.csv

```
# 1. Read the students.csv file.
import csv

with open("students.csv", "r") as file:
    data = csv.reader(file)
    for row in data:
        print(row)

# 2. Remove students whose marks are less than 70.
clean_data = []
with open("students.csv", "r") as file:
    read_data = csv.DictReader(file)
    for row in read_data:
        if int(row['marks']) > 70:
            number = row.copy()
            number['name'] = number['name'].upper()
            clean_data.append(number)

# 3. Convert student names to uppercase.
# 4. Save the cleaned data into: clean_students.csv
with open("clean_students.csv", "w") as file:
    writer = csv.DictWriter(file, fieldnames= ['id', 'name', 'marks', 'city'])
    writer.writeheader()
    writer.writerows(clean_data)

with open("clean_students.csv", "r") as file:
    read_data = csv.DictReader(file)
    for row in read_data:
        print(row)
```

Output :

```
{'id': '1', 'name': 'RAHUL', 'marks': '85', 'city': 'Mumbai'}
{'id': '2', 'name': 'SNEHA', 'marks': '78', 'city': 'Pune'}
{'id': '3', 'name': 'AMIT', 'marks': '90', 'city': 'Delhi'}
{'id': '5', 'name': 'ROHIT', 'marks': '88', 'city': 'Mumbai'}
{'id': '6', 'name': 'ANJALI', 'marks': '72', 'city': 'Pune'}
{'id': '7', 'name': 'KARAN', 'marks': '95', 'city': 'Delhi'}
{'id': '9', 'name': 'VIJAY', 'marks': '81', 'city': 'Mumbai'}
{'id': '10', 'name': 'POOJA', 'marks': '75', 'city': 'Pune'}
{'id': '12', 'name': 'NEHA', 'marks': '84', 'city': 'Nashik'}
{'id': '13', 'name': 'SANJAY', 'marks': '77', 'city': 'Mumbai'}
{'id': '14', 'name': 'KAVITA', 'marks': '91', 'city': 'Pune'}
{'id': '15', 'name': 'DEEPAK', 'marks': '73', 'city': 'Delhi'}
```

STEP 4 – Linux Tasks

Using terminal commands:

1. Count number of lines in students.csv.
2. Display first 3 lines of the file.
3. Search for students from city Pune.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/student_data_project$ cat students.csv | wc -l
16
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/student_data_project$ head -n 5 students.csv | column -t -s,
id  name  marks  city
1   Rahul  85     Mumbai
2   Sneha  78     Pune
3   Amit   90     Delhi
4   Priya  65     Nashik
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/student_data_project$ grep -i "puNE" students.csv
2,Sneha,78,Pune
6,Anjali,72,Pune
10,Pooja,75,Pune
14,Kavita,91,Pune
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/student_data_project$ |
```

STEP 5 – SQL Analysis

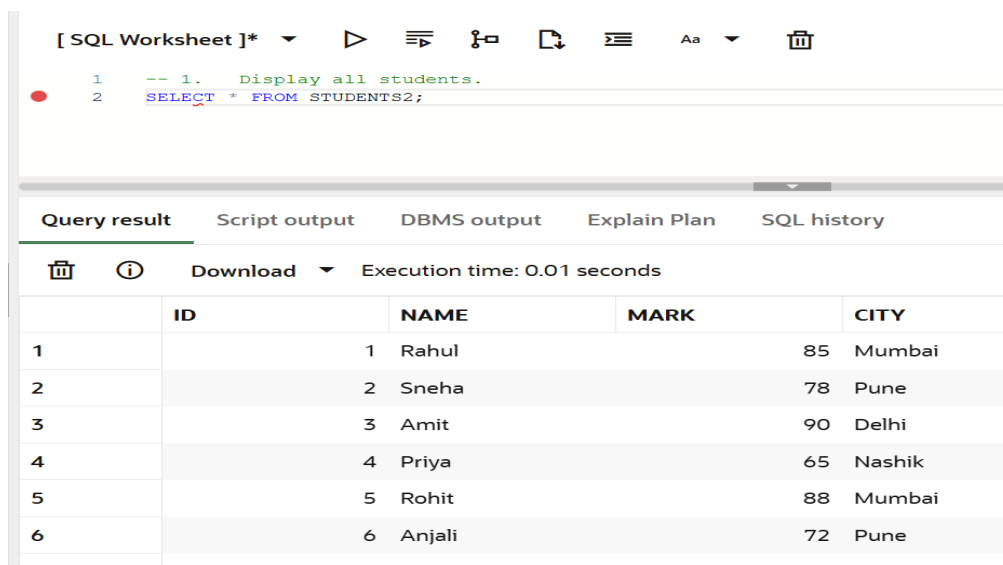
Create a table called students.

Columns : Id, name, city, marks

Insert the cleaned data from clean_students.csv.

Write SQL queries to answer the following:

1. Display all students.



The screenshot shows a SQL worksheet with a query editor at the top and a results table below. The query editor contains two lines: a comment and a SQL query. The results table has five columns: ID, NAME, MARK, and CITY. It displays seven rows of student data.

ID	NAME	MARK	CITY
1	Rahul	85	Mumbai
2	Sneha	78	Pune
3	Amit	90	Delhi
4	Priya	65	Nashik
5	Rohit	88	Mumbai
6	Anjali	72	Pune
7	Kavita	91	Delhi

2. Find average marks.

```
6  -- 2.    Find average marks.
7  SELECT AVG(MARK) AS AVG_MARK FROM STUDENTS2;
```

Query result Script output DBMS output Explain Pla



Download ▼

Execution time: 0.004 seconds

	AVG_MARK
1	78.9

3. Find student with highest marks.

```
6  -----
7  -- 3.    Find student with highest marks.
8  SELECT NAME, MARK FROM STUDENTS2 ORDER BY MARK DESC FETCH FIRST 1 ROW ONLY;
9
```

Query result Script output DBMS output Explain Plan SQL history

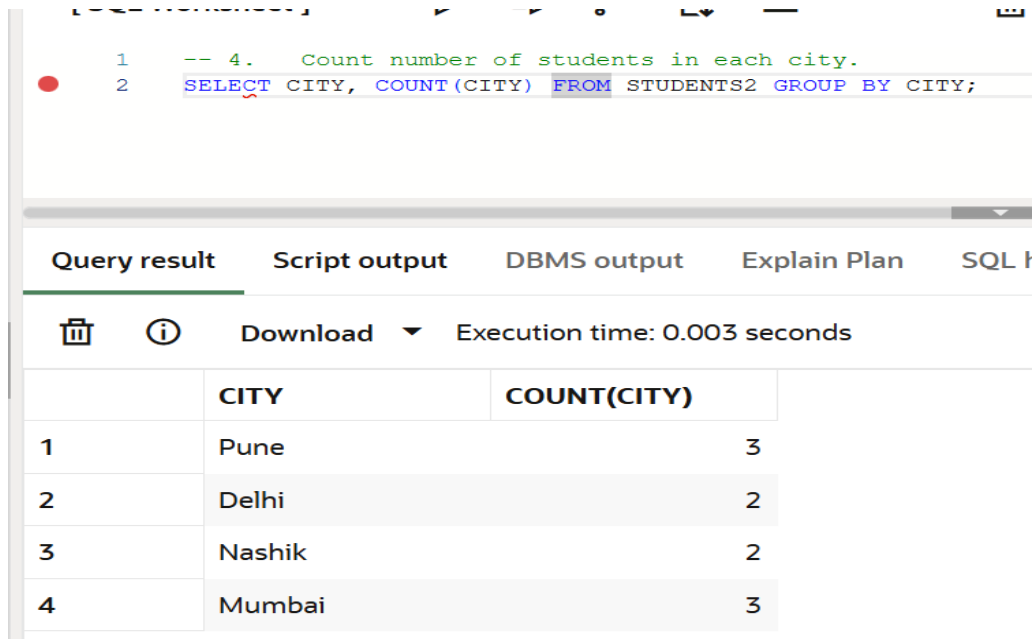


Download ▼

Execution time: 0.007 seconds

	NAME	MARK
1	Karan	95

4. Count number of students in each city.



The screenshot shows a SQL query execution interface. At the top, there is a query editor with two lines of code: a comment and a SELECT statement. Below the editor, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL H'. The 'Query result' tab is selected, showing a table with 4 rows and 2 columns: 'CITY' and 'COUNT(CITY)'. The table contains data for Pune, Delhi, Nashik, and Mumbai. Below the table, there is a 'Download' button and a status message 'Execution time: 0.003 seconds'.

```
1  -- 4.    Count number of students in each city.
2  SELECT CITY, COUNT(CITY) FROM STUDENTS2 GROUP BY CITY;
```

	CITY	COUNT(CITY)
1	Pune	3
2	Delhi	2
3	Nashik	2
4	Mumbai	3

Download Execution time: 0.003 seconds